

Embeddings Explained

From Word2Vec to Modern Retrieval

data object



embedding model



data as vector embedding

0.056	0.009	-0.07	...	0.023
-------	-------	-------	-----	-------

-0.06	0.039	0.137	...	0.045
-------	-------	-------	-----	-------

0.012	-0.01	0.079	...	-0.02
-------	-------	-------	-----	-------



AI Coach John

Follow



🔄 Repost

What Is an Embedding?

Turning meaning into geometry

An embedding is a list of numbers (a vector) representing text in a way that captures its **meaning**: so that texts with similar meanings end up close together in vector space.

VECTOR SPACE PROXIMITY

```
"king"    → [ 0.21, -0.44,  0.87, ... ] (Close)
"queen"   → [ 0.19, -0.41,  0.85, ... ] (Close)
"monarch" → [ 0.22, -0.43,  0.88, ... ] (Close)
-----
"banana"  → [-0.71,  0.33, -0.12, ... ] (Far Away)
```

2D Semantic Cluster Projection



Royal Cluster

king • queen • monarch



Fruit Cluster

banana • apple



Why it matters: Computers can't compare meanings, but they can compare vector numbers instantly at massive scale.



AI Coach John

Follow



 Repost

The Roadmap: 6 Eras

From counting words to understanding them

1. One-hot & Bag-of-Words

Exact index matching, no meaning

1960s-2000s

2. TF-IDF

Word importance weighting, no semantics

1970s+

3. Word2Vec / GloVe

Semantic vectors, static per word

2013-2014

4. ELMo & BERT

Contextual embeddings, bidirectional

2018

5. Sentence-BERT (SBERT)

Siamese bi-encoders for full sentences

2019

6. Modern Retrieval Models

Multilingual, instructions & Matryoshka

2022-Now



AI Coach John

Follow



 Repost

Era 1: One-Hot & BoW

Where it started: text as pure counting

ONE-HOT VECTOR REPRESENTATION

```
vocab = ["cat", "dog", "king", "queen"]
```

```
"cat" → [1, 0, 0, 0]
```

```
"king" → [0, 0, 1, 0]
```

```
"queen" → [0, 0, 0, 1]
```

```
# Similarity calculation:
```

```
dot([0,0,1,0], [0,0,0,1]) = 0 # Zero similarity!
```

Vocabulary Size: 50,000 Words

```
"cat" → [0, 0, 0, ..., 1, ..., 0, 0, 0]
```

50,000 dimensions (49,999 zeros)



The Fatal Flaw: Every word is equidistant from every other word.

"King" and "queen" are treated as unrelated as "king" and "banana."



Era 2: TF-IDF

Weighting words by how informative they are

TF-IDF FORMULA

$\text{TF-IDF} = (\text{Term Frequency in doc}) \times (\text{Inverse Document Frequency across corpus})$

Document: "the embedding model is fast"

"the"		0.01
"embedding"		0.89
"model"		0.52

✓ Fixed

Filters out common filler words ("the", "is")

✗ Still Broken

"Car" and "automobile" remain 0% match



Why BM25 still matters: BM25 (refined TF-IDF) is used in hybrid search for exact matches like serial codes & names.



AI Coach John

Follow



Repost

Era 3: Word2Vec

Meaning comes from context (The Breakthrough)

The Distributional Hypothesis

Words that appear in similar contexts have similar meanings. "Dog" and "cat" both appear near "pet", "feed", "vet" so their vectors align.

Skip-Gram Training Architecture

"the fluffy [CAT] sat on the mat"

Input: "cat" → **Predict:** ["fluffy", "sat", "on"]

The model isn't kept for predictions: the hidden layer weights ARE the embeddings!

Skip-gram: Given a word, predict context neighbors.

CBOW: Given context, predict center word.



Word2Vec: Vector Math

Proof that vectors encode real semantic structure

VECTOR ARITHMETIC EXAMPLES

$$\text{vec}(\text{"king"}) - \text{vec}(\text{"man"}) + \text{vec}(\text{"woman"}) \approx \text{vec}(\text{"queen"})$$
$$\text{vec}(\text{"paris"}) - \text{vec}(\text{"france"}) + \text{vec}(\text{"italy"}) \approx \text{vec}(\text{"rome"})$$

Consistent Geometric Relationships

king → queen (Gender Offset)

man → woman (Gender Offset)



Remaining Limitation: One fixed vector per word. "Bank" gets a single vector averaging *river bank* and *money bank*.



AI Coach John

Follow



PROTBRIDGE
Strives For Better Future



Repost

Era 4: Contextual BERT

Vectors change depending on context

Solving the Polysemy Problem

"Sat by the river **bank**" → [0.31, -0.22, ...]

"Deposit cash at **bank**" → [-0.64, 0.51, ...]

(Completely different vectors generated per context)

⚙️ BERT Self-Supervised Objectives

- ✓ **Masked Language Model (MLM):** Hide ~15% of tokens and predict from both left & right context
- ✓ **Next Sentence Prediction (NSP):** Predict whether sentence B follows sentence A



Bidirectional context allows BERT to understand exact word meaning in full sentences.



Why Raw BERT Fails Search

The problem that created a whole new model class

Raw BERT = Cross-Encoder Only

BERT outputs a vector per token, not per sentence. Comparing a query to 10,000 docs requires feeding both together 10,000 times: far too slow for real-time search!

Cross-Encoder (Raw BERT)

[query + doc] → model → score
10,000 passes / query (Too Slow)

Desired Bi-Encoder

[query] → vec | [doc] → vec
Precompute docs once!



Averaging raw BERT token vectors yields poor semantic similarity performance.



Era 5: Sentence-BERT

Making sentence embeddings truly comparable

Siamese Twin-Tower Network

Sentence A → [BERT Tower]
→ Vector A

Sentence B → [BERT Tower]
→ Vector B

Compute Cosine Similarity (Fast Dot Product)

🎯 Contrastive Learning Recipe

Anchor: "How to reset password?"

Positive: "Process for password recovery" → Pull Closer

Negative: "Shipping rates info" → Push Apart



Allows precomputing 10,000 document vectors once, reducing query time to milliseconds.



AI Coach John

Follow



 Repost

★ ERA 6 (2022-NOW)

Era 6: Modern Models

What has changed since SBERT



Hard negative mining



Instruction-aware prefixes



100+ Multilingual spaces



8,192+ token context

Feature	SBERT (2019)	Modern Models
Context Limit	512 tokens	8,192+ tokens
Languages	English-focused	100+ Multilingual
Dimensions	Fixed size	Matryoshka (Truncatable)



Popular options today: OpenAI text-embedding-3, BGE, E5, Nomic, Cohere Embed.



AI Coach John

Follow



 Repost

Instruction-Tuned Embeds

Telling the model what kind of similarity you want

TASK PREFIX USAGE

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("BAAI/bge-small-en-v1.5")

prefix = "Represent this sentence for searching relevant passages: "
q_vec = model.encode(prefix + "How do I reset my password?")
doc_vec = model.encode("Visit settings to recover your account.")
```

✗ Without Prefix (Surface Match):

"How to reset password?" ≈ "How to reset router?" (Irrelevant)

✓ With Prefix (Asymmetric RAG):

"How to reset password?" ≈ "Visit settings account recovery" (Useful)



Always check model docs: omitting a required prefix measurably degrades retrieval.



Computing Similarity

The math behind "these are close"

COSINE SIMILARITY MATH

```
def cosine_similarity(a, b):  
    return np.dot(a, b) / (np.linalg.norm(a) *  
        np.linalg.norm(b))
```

```
# If vectors are normalized (length 1):  
similarity = np.dot(a, b)
```

Angle Interpretation between Vectors

Small Angle ($\theta \approx 0^\circ$)
→ Similar (≈ 1.0)

90° Angle
→ Unrelated (≈ 0.0)

180° Opposite
→ Opposite (≈ -1.0)



Pro Tip: Always normalize embeddings. Then simple dot product gives cosine similarity instantly.



AI Coach John

Follow



PROTBRIDGE
Strive For Better Futures



Repost

Bi vs Cross vs ColBERT

Three points on the speed and accuracy curve

SPEED (Fastest)

Bi-Encoder

1 vec/doc (Fast)

BALANCE

ColBERT

Late Interaction

ACCURACY (Highest)

Cross-Encoder

Full Attention



Production Pattern: Bi-encoder retrieves top 50 → Cross-encoder reranks to top 5.



AI Coach John

Follow



 Repost

Choosing a Model

What actually matters when picking one

Factor	What to consider
Dimensions	Higher = more nuance, but more storage (384-768 sweet spot)
Max Input Length	Must exceed your chunk size to prevent silent truncation
Domain Fit	Check performance on legal, medical, or code benchmarks
Local vs API	Local (BGE, Nomic) = private, free. API (OpenAI) = zero infra



Golden Rule: Benchmark 2-3 candidate models on *your* actual domain queries & corpus!



AI Coach John

Follow



 Repost

Critical Rules to Follow

The mistakes that silently break retrieval

✖ Never mix models

Vectors from different models live in incompatible spaces.

↻ Model change = Re-embed

Upgrading a model requires regenerating your entire vector index.

🔗 Match Query/Doc Encoder

Use identical instruction prefixes for query and documents.

⚠ Watch Input Length

Text past context limit gets silently truncated without warning.

🔪 Normalize Consistently

Normalize vectors at both index and query time.

🗣 Semantic ≠ Factually True

Opposite factual statements can still rank high by topic.



Beyond Text Embeddings

The same idea, applied everywhere



Image Embeds (CLIP)



Code Embeddings



Audio & Speech Embeds



Graph & Entity Embeds

Unified Vector Space Mapping

Text / Image / Code →

Shared Vector Space

→ Geometric Search



Anything with a similar/dissimilar signal can be embedded and searched geometrically.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future



Repost

The Evolution Recapped

Every era and what it solved

Era	Approach	What it solved	Limitation
One-hot	Identity index	Numeric text	No meaning
TF-IDF	Weighted counts	Word importance	No semantics
Word2Vec	Predict context	Semantic vectors	1 vector/word
BERT	Bi-directional	Contextual meaning	Not comparable
SBERT	Siamese bi-encoder	Fast sentence math	Short context
Modern	Instruction + Scale	Long, multilingual	Needs domain test



 CORE TAKEAWAY

Every RAG system stands on this

From counting words to instruction-aware retrieval models: embeddings let computers answer "what does this mean?" as a geometry problem.

One-Hot → TF-IDF → Word2Vec → BERT → SBERT → [Modern Retrieval](#)

Save this reference guide

Follow for more AI engineering breakdowns



AI Coach John

Follow



 **Repost**